

Python Threading — Beginner Notes

"Doing Multiple Things at the Same Time"

Before We Start — What Do You Already Need to Know?

Just these basics:

- How to write a Python function (`def`)
- How to use a `for` loop
- How to use `import`
- What `time.sleep()` does (pauses the program)

That's it. No prior knowledge of threading needed.

1. The Problem — Python Is Normally "One Thing at a Time"

By default, Python does tasks **one after another**. It finishes the first task completely before moving to the second.

Think of it like a single chef in a kitchen. He boils water, waits for it to boil, then cuts vegetables, then fries them. Everything happens one after another — even if waiting for water to boil means standing there doing nothing.

Here's what that looks like in code:

```
import time

def download_file(name):
    print(f"Downloading {name}...")
    time.sleep(3)           # pretend this takes 3 seconds
    print(f"{name} downloaded!")

download_file("file_1.txt") # waits 3 seconds
download_file("file_2.txt") # waits another 3 seconds
download_file("file_3.txt") # waits another 3 seconds

print("All done!")
```

Total time: 9 seconds.

But here's the thing — while `file_1.txt` is "downloading" (sleeping for 3 seconds), Python is just sitting there doing nothing. It's not busy. It's just waiting. What if we could download all 3 files at the same time?

That's exactly what **threading** lets us do.

2. What Is a Thread?

A **thread** is like an independent worker that runs your function separately, in the background.

When you create a thread and start it, Python doesn't wait for it to finish. It just starts the thread and immediately moves on to the next line of code. The thread runs on its own.

Real life analogy: Imagine you are the manager. You tell Worker 1 to download file 1, Worker 2 to download file 2, Worker 3 to download file 3 — all at the same time. Instead of doing everything yourself one by one, all three workers do their jobs simultaneously.

3. Your First Thread — Step by Step

Python has a built-in module called `threading`. Let's use it.

```
import threading
import time

def download_file(name):
    print(f"Downloading {name}...")
    time.sleep(3)
    print(f"{name} downloaded!")

# Create a thread for each file
t1 = threading.Thread(target=download_file, args=("file_1.txt",))
t2 = threading.Thread(target=download_file, args=("file_2.txt",))
t3 = threading.Thread(target=download_file, args=("file_3.txt",))

# Start all three threads
t1.start()
t2.start()
t3.start()

# Wait for all threads to finish
t1.join()
t2.join()
t3.join()

print("All done!")
```

Total time: ~3 seconds (instead of 9).

Let's break down what each line does:

`threading.Thread(target=..., args=(...))`

This **creates** a thread. It doesn't run anything yet.

- `target` — the function you want the thread to run

- `args` — the arguments to pass to that function (must be a **tuple**, so add a comma at the end even for one argument)

```
t1 = threading.Thread(target=download_file, args=("file_1.txt",))
#                               ↑ the function           ↑ arguments as a tuple
```

`.start()`

This **starts** the thread. The function now runs in the background.

```
t1.start()    # download_file("file_1.txt") starts running in the background
              # Python does NOT wait here – it immediately moves to the next line
```

`.join()`

This tells the main program: **"wait here until this thread finishes."**

Without `.join()`, the main program might print "All done!" before the threads even finish their work.

```
t1.join()    # wait until t1 is completely done
t2.join()    # wait until t2 is completely done
t3.join()    # wait until t3 is completely done
print("All done!")    # only runs after all three are done
```

4. Passing Arguments to a Thread

You can pass any number of arguments using `args`:

```
import threading
import time

def greet(name, times):
    for i in range(times):
        print(f"Hello, {name}! (message {i+1})")
        time.sleep(1)

t = threading.Thread(target=greet, args=("Rahul", 3))
t.start()
t.join()
```

For keyword arguments, use `kwargs`:

```
t = threading.Thread(target=greet, kwargs={"name": "Rahul", "times": 3})
t.start()
```

```
t.join()
```

5. Running Multiple Threads Together — A Cleaner Way

When you have many threads, use a list to manage them:

```
import threading
import time

def task(task_number):
    print(f"Task {task_number} started")
    time.sleep(2)
    print(f"Task {task_number} finished")

# Create 5 threads
threads = []
for i in range(1, 6):
    t = threading.Thread(target=task, args=(i,))
    threads.append(t)

# Start all threads
for t in threads:
    t.start()

# Wait for all threads to finish
for t in threads:
    t.join()

print("All tasks done!")
```

Output will look something like this (order may vary):

```
Task 1 started
Task 2 started
Task 3 started
Task 4 started
Task 5 started
Task 1 finished
Task 2 finished
...
All tasks done!
```

6. The Shared Variable Problem — Race Condition

Here is something very important that beginners often miss.

When two threads **share the same variable**, they can accidentally overwrite each other's work. This is called a **race condition** — and it causes bugs that are very hard to find because they don't happen every

time.

Let's see an example:

```
import threading

counter = 0 # shared variable

def add_one():
    global counter
    for _ in range(1000000):
        counter = counter + 1 # both threads do this at the same time

t1 = threading.Thread(target=add_one)
t2 = threading.Thread(target=add_one)

t1.start()
t2.start()

t1.join()
t2.join()

print(f"Counter value: {counter}")
# Expected: 2000000
# Actual: something like 147823 – different every time!
```

Why does this happen?

When two threads both read the counter at the same time (say, both see `counter = 500`), both add 1 and write back `501`. But the correct answer should be `502`. One increment is lost.

This is like two people editing the same Google Doc word at the same time — they overwrite each other.

7. The Fix — `threading.Lock()`

A **Lock** is like a key to a room. Only one person (thread) can hold the key at a time. Others have to wait outside until the key is returned.

```
import threading

counter = 0
lock = threading.Lock() # create one lock

def add_one():
    global counter
    for _ in range(1000000):
        lock.acquire() # pick up the key – other threads must wait
        counter = counter + 1
        lock.release() # put the key back – next thread can proceed

t1 = threading.Thread(target=add_one)
t2 = threading.Thread(target=add_one)
```

```

t1.start()
t2.start()

t1.join()
t2.join()

print(f"Counter value: {counter}")    # Now always: 200000

```

Better way — use `with lock:` (recommended always)

The `with` statement automatically acquires and releases the lock — even if an error happens inside:

```

def add_one():
    global counter
    for _ in range(100000):
        with lock:                # acquires lock here
            counter += 1          # only one thread runs this at a time
                                # lock released automatically when block ends

```

Simple rule: Any time two or more threads read AND write the same variable, protect it with a `Lock`.

8. `threading.Event()` — One Thread Signals Another

Sometimes one thread needs to wait for another thread to finish something before it starts working. An **Event** is like a green light — one thread turns it green, and all waiting threads can then proceed.

```

import threading
import time

# Create an event (starts as "red light" — not set)
ready_event = threading.Event()

def setup_worker():
    print("Setting up... please wait")
    time.sleep(3)                # simulates setup taking time
    print("Setup complete!")
    ready_event.set()            # turn on the green light

def main_worker():
    print("Main worker waiting for setup...")
    ready_event.wait()           # pause here until the event is set
    print("Setup is done! Main worker starting now.")

t1 = threading.Thread(target=setup_worker)
t2 = threading.Thread(target=main_worker)

t2.start()    # starts waiting immediately
t1.start()    # starts doing setup

```

```
t1.join()
t2.join()
```

Output:

```
Main worker waiting for setup...
Setting up... please wait
Setup complete!
Setup is done! Main worker starting now.
```

9. `queue.Queue` — Safe Way to Pass Data Between Threads

A **Queue** is like a conveyor belt between threads. One thread puts items on it, another thread picks items off it. The Queue handles all the safety and waiting automatically — you don't need a Lock.

```
import threading
import queue
import time

def producer(q):
    """Puts items into the queue."""
    for i in range(5):
        item = f"item_{i}"
        q.put(item)          # add to the queue
        print(f"[Producer] Put {item}")
        time.sleep(0.5)

def consumer(q):
    """Takes items from the queue."""
    for i in range(5):
        item = q.get()        # take from queue — waits if queue is empty
        print(f"[Consumer] Got {item}")
        q.task_done()        # tell the queue this item is processed

# Create a queue
q = queue.Queue()

t1 = threading.Thread(target=producer, args=(q,))
t2 = threading.Thread(target=consumer, args=(q,))

t1.start()
t2.start()

t1.join()
t2.join()
```

Output:

```
[Producer] Put item_0
[Consumer] Got item_0
```

```
[Producer] Put item_1
[Consumer] Got item_1
...
```

Limiting queue size

You can limit how many items the queue holds at once. If it's full, the producer will wait automatically:

```
q = queue.Queue(maxsize=3)    # holds max 3 items at a time
```

10. Daemon Threads — Background Workers That Auto-Stop

By default, Python will NOT exit until all threads have finished. Sometimes you want a background thread that just runs forever (like a sensor polling loop) and should automatically stop when the main program ends.

That's what `daemon=True` is for.

```
import threading
import time

def background_sensor():
    while True:
        print("Reading sensor...")
        time.sleep(1)

# daemon=True - this thread dies automatically when the main program exits
t = threading.Thread(target=background_sensor, daemon=True)
t.start()

# Main program does its work
print("Main program running...")
time.sleep(3)
print("Main program done. Exiting.")
# The daemon thread is automatically killed here - no need to stop it manually
```

Without `daemon=True`, the program would run forever because the `while True` thread never stops.

11. Getting a Return Value from a Thread

Threads don't return values directly. The easiest beginner approach is to use a list or dictionary to collect results:

```
import threading

results = {}    # shared dictionary to store results

def square(number):
```



```

results[number] = number * number

threads = []
for n in [2, 4, 6, 8, 10]:
    t = threading.Thread(target=square, args=(n,))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print(results)
# {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}

```

12. Quick Reference Card

Thing	Code	What it does
Create a thread	<code>t = threading.Thread(target=fn, args=(x,))</code>	Prepares a thread
Start a thread	<code>t.start()</code>	Starts running the function
Wait for a thread	<code>t.join()</code>	Pauses main program until thread is done
Create a lock	<code>lock = threading.Lock()</code>	Creates a key for protecting shared data
Use a lock	<code>with lock: ...</code>	Only one thread at a time in this block
Create an event	<code>e = threading.Event()</code>	Creates a green/red light signal
Signal the event	<code>e.set()</code>	Turns light green
Wait for the event	<code>e.wait()</code>	Pauses until light turns green
Create a queue	<code>q = queue.Queue()</code>	Conveyor belt between threads
Add to queue	<code>q.put(item)</code>	Put item on conveyor belt
Take from queue	<code>q.get()</code>	Take item off conveyor belt
Background thread	<code>daemon=True</code>	Auto-kills when main program exits

13. When Should You Use Threading?

Use threading when your program spends time WAITING:

- Downloading files from the internet
- Reading from a camera or sensor
- Making API calls
- Writing/reading files

Do NOT expect threading to speed up:

- Heavy math calculations done purely in Python (like a `for` loop calculating numbers)
- Tasks that don't involve any waiting at all

Common Beginner Mistakes

Mistake	Fix
Forgetting <code>.start()</code> after creating a thread	Always call <code>.start()</code> — creating a thread doesn't run it
Forgetting <code>.join()</code>	Your main program might finish before threads do — always <code>join()</code>
Modifying shared variables without a lock	Always use <code>with lock:</code> when multiple threads share a variable
Passing arguments without a trailing comma	<code>args=("hello",)</code> — that comma is required for single-item tuples

Summary Checklist

- ☐ I can explain what a thread is in my own words
- ☐ I can create a thread with `threading.Thread(target=..., args=(...))`
- ☐ I can start a thread with `.start()` and wait for it with `.join()`
- ☐ I know what a race condition is and can recognise one
- ☐ I can fix a race condition using `with lock:`
- ☐ I can use `threading.Event()` to signal between threads
- ☐ I can use `queue.Queue` to pass data between two threads
- ☐ I know what `daemon=True` does and when to use it